CAIRN: Provenance and Integrity for the Regulated Knowledge Base

A Zero-Dependency, Local-First System for Cite-or-Refuse Answering, Tamper-Evident Receipts, and Corpus-Level Contradiction Surveillance

JourdanLabs Technical White Paper · Version 1.0 · July 2026

Leland Jourdan II, Founder & Chief Architect, JourdanLabs · `leland@jourdanlabs.com`

Abstract

Background. In regulated firms, the internal knowledge base functions as a *control surface*: policies, procedures, and control narratives are the mechanism by which staff — and, increasingly, AI assistants — determine correct action. This surface degrades along four axes: internal inconsistency, staleness, ungrounded retrieval, and un-auditability. Retrieval-augmented AI assistants layered onto such a corpus do not mitigate these failures; they amplify their consequences by producing fluent, confident answers with no provenance and no artifact tying an answer to its source.

Problem. Existing tools optimize for the wrong question. Enterprise search optimizes *findability*; document-AI assistants optimize *fluency*. Neither verifies that the corpus agrees with itself, and neither emits a verifiable record of where a given answer came from. The gap is not retrieval quality — it is **provenance and integrity**.

Approach. CAIRN is a zero-runtime-dependency, local-first system built on three mechanisms: (i) *cite-or-refuse* answering in which a confidence gate refuses weak matches before any generative model is invoked, so the model cannot hallucinate to fill a gap; (ii) a *hash-chained receipt ledger* that seals every answer and every refusal into an append-only, tamper-evident record with per-source SHA-256 content hashes; and (iii) *corpus-level integrity surveillance* combining a deterministic structural audit with embedding-based overlap detection and model adjudication of contradiction, duplication, or relation, rolled into a single hashed integrity artifact.

Properties. Determinism where the corpus is audited; a receipt on every answer and every refusal; tamper-evidence by construction; and full local operation in which the corpus never leaves the machine. The implementation carries no third-party runtime dependencies (Node.js built-ins only), is covered by 53 automated tests, and was independently re-executed — not accepted on assertion — including a tamper-evidence stress test in which three distinct edits to a sealed record were each detected with the correct cause.

Status. Code-complete for all capabilities exercisable without a customer's environment. Capabilities that require the customer's own systems (a live tenant token, an identity provider, in-perimeter deployment, external certification) are config-driven and coded to real vendor APIs but are, by definition, not run here — a boundary stated explicitly rather than obscured.

Keywords. Knowledge integrity, answer provenance, retrieval-augmented generation, cite-or-refuse, tamper-evident receipts, contradiction detection, regulated AI, local-first systems, model-risk governance.

1. Introduction

1.1 The knowledge base as a control surface

In a regulated institution, documentation is not an incidental artifact — it is the substrate of control. A control narrative states how a risk is mitigated; a procedure states how a task is performed; a policy states what is permitted. When staff act, they act on what the knowledge base tells them. The correctness of the institution's behavior is therefore bounded by the *integrity* of that corpus: whether it is internally consistent, current, and reachable, and whether an action can be traced to the source that informed it.

1.2 Why AI assistants amplify rather than mitigate

Layering a retrieval-augmented assistant onto the corpus improves *access latency* but degrades *auditability*. A conventional assistant retrieves the nearest-sounding passages and generates a fluent synthesis. Two failure modes follow directly. First, fluency is mistaken for correctness: an answer drawn from a superseded or incorrect document is indistinguishable, in tone, from a correct one. Second, the answer carries no artifact: when a regulator later asks how a decision was informed, there is no record binding the decision to a source. The more heavily the institution relies on "chat with your documents," the larger this un-auditable surface grows — and it grows fastest in exactly the environments least able to absorb the resulting risk.

1.3 The provenance gap

The deficiency is not addressed by better retrieval. A more accurate retriever still produces an un-receipted answer and still says nothing about whether the corpus contradicts itself. What is missing is a system whose *output contract* is provenance: every answer either proves its sourcing or refuses, and the corpus itself is continuously interrogated for internal conflict. This reframes the problem from "answer the question well" to "answer only what can be proven, and know whether the knowledge base agrees with itself."

1.4 Contributions

This paper describes CAIRN, a system contributing: (1) a *refusal-first* answering discipline with a pre-generation confidence gate; (2) a *hash-chained receipt* for every answer and refusal, with per-source content hashing; (3) a *deterministic-plus-adjudicated* integrity model that separates cheap structural defects from model-confirmed contradictions; (4) an in-process approximate-nearest-neighbor design that scales contradiction candidate-generation without an external vector store; and (5) an engineering posture — zero runtime dependencies, local-first, tamper-evident — chosen so the system is deployable inside a regulated perimeter and auditable end-to-end. We also state, explicitly, the boundaries the system does not cross.

2. Threat Model and Design Principles

CAIRN's design assumes an adversary is not primarily malicious but *entropic*: the corpus drifts, documents supersede one another silently, and well-intentioned answers are sourced from the wrong place. It additionally assumes an auditor who will later ask, adversarially, to reconstruct how an answer was produced and whether any record was altered after the fact. Five principles follow.

1. **Refuse over guess.** The confidence gate rejects weak retrievals *before* a generative model is invoked. "Not in your vault" is a first-class, receipted outcome.
2. **Everything checkable.** Every answer and report re-verifies from its hash; there is no black box the customer cannot audit.
3. **Deterministic where it can be.** Retrieval ranking, the structural audit, and the integrity score are reproducible given the same index.
4. **Local by default.** Corpus and models can run fully air-gapped; nothing leaves the perimeter unless explicitly configured to.
5. **State the boundary.** Where a capability cannot be exercised without the customer's environment, that is documented, not implied to be complete.

---

3. System Architecture

```
sources ──► CONNECTORS ──► INGESTION ──► STORES ──► ENGINES ──► API ──► clients
(filesystem,  (normalized    (parse →     (docs,     (hybrid       (REST,
 Confluence,   Document       chunk →      vectors,   retrieval,    RBAC/OIDC,
 REST)         model)         index)       receipt    grounded      audit,
                                           ledger,    answer,       compliance
                                           reports)   audit,        export)
                                                      surveillance)
```

- **Connectors** normalize any source into one `Document` shape, so ingestion, stores, and engines never depend on where a document originated.
- **Ingestion** parses frontmatter, headings, `#tags`, and `[[wikilinks]]`, chunks by heading, and builds an in-memory index driving lexical (BM25) and, when embeddings are available, hybrid retrieval.
- **Stores** comprise the document/chunk index, an optional vector index, and a hash-chained receipt ledger persisted on disk.
- **Engines** provide hybrid retrieval, grounded answering, the deterministic audit, contradiction surveillance, and integrity scoring.
- **API** exposes the above over REST with role-based authorization and full request gating.

The implementation is pure Node.js (≥18) using built-ins only, with **no third-party runtime dependencies**: there is nothing to `npm install`, patch, or supply-chain-audit, and the system starts from a clean checkout with `node server.mjs`.

---

4. The Receipt and the Hash-Chained Ledger

4.1 Canonical hashing

All hashing operates over a canonical JSON serialization in which object keys are sorted recursively and `undefined` values are dropped, so two payloads differing only in key order hash identically. Content hashes are SHA-256.

4.2 The answer/report receipt

Every answer and every refusal emits a receipt recording the question, the verdict (`GROUNDED` or `REFUSED_UNGROUNDED`), each cited source **content-hashed with SHA-256** (a proof of exactly what the source said at answer time), the model, the confidence, the index build time, a timestamp, and a `receipt_sha256` over the whole. The integrity report emits an analogous `report_sha256`. Both are downloadable JSON artifacts.

4.3 The ledger seal

Receipts are appended to a hash-chained, append-only log ("the LUNA pattern"). Each entry binds its sequence number, the prior entry's hash, its kind, and the canonical payload:

```
entry_hash = SHA256( seq | prev_hash | kind | canonicalJSON(payload) )
```

A verification pass walks the chain, recomputing each seal and checking prior-hash linkage; any edit to a hashed field of any past entry — sequence, linkage, or payload — and any deletion or reordering of a line is detected. The verify result reports the first broken index and the reason.

4.4 Honest properties of the ledger

Two properties are stated plainly. First, the human-readable `ts` timestamp is deliberately excluded from the hash (so reproducible hashes do not require a fixed clock); consequently a change to `ts` alone is the one edit the chain does not flag. Second, the chain is correct for a single writer; two processes appending to the same ledger file concurrently can fork it, so deployments run one writer per ledger. Neither property is hidden; both are documented where the mechanism is described.

---

5. Grounded, Cite-or-Refuse Answering

Retrieval combines lexical BM25 with semantic similarity when an embedding model is configured (hybrid), and lexical-only otherwise; the mode is surfaced to the user rather than assumed. A **confidence gate** evaluates retrieval strength *before* any generative call. On a weak or empty result the system refuses — returning the closest passages and a receipt — and never invokes the model to fill the gap. On a sufficient result the model is constrained to answer from the retrieved contexts, with citations; the answer, its citations, and its receipt are returned together.

Refusal is a designed outcome, not an error path: a wrong-but-confident answer is precisely the failure the system exists to prevent, so a receipted "Not in your vault" is treated as success. Because both branches are receipted, the *rate and provenance of refusals* are themselves auditable.

---

6. Integrity and Contradiction Surveillance

6.1 Deterministic structural audit

A deterministic pass identifies orphaned notes (no inbound or outbound links), broken `[[links]]`, stale notes (beyond a staleness horizon with open tasks or draft markers), duplicate titles (retrieval ambiguity), stubs, and untagged notes. These are reproducible given the same index and carry deterministic penalties.

6.2 Overlap detection and adjudication

Embedding-based overlap detection surfaces cross-note chunk pairs above a similarity threshold, deduplicated to the strongest pair per note-pair. Where a generative model is configured, an adjudication step classifies each candidate as **contradict**, **duplicate**, **related**, or **unrelated**. The system is explicit that similarity yields *candidates*, and only adjudication yields a *verdict*.

6.3 Controlled sources

Documents may be marked *controlled* (authoritative) by folder or frontmatter. The integrity report reports controlled coverage, and same-topic overlaps between two controlled documents are escalated as **review priorities** — a genuine conflict between two authoritative policies is the costliest class of defect and is surfaced first.

6.4 Scoring philosophy

The integrity score (0–100 with a letter grade) is deliberately conservative and defensible: structural defects incur deterministic penalties; unadjudicated contradiction candidates incur only a light penalty; and a heavy penalty is applied to a controlled-versus-controlled contradiction **only once adjudication confirms it**. The score, its constituent findings, and the contradiction candidates roll into one hashed (`report_sha256`) artifact.

6.5 Surveillance over time

Surveillance snapshots the integrity state, diffs it against the prior sealed baseline, and alerts on **new** defects and newly confirmed contradictions only — the first run is a silent baseline — via webhook, file, or SIEM sinks. (In the current build this executes on demand per request; an interval scheduler is implemented but not yet wired to run as a background loop — see §11.)

---

7. Scale

The naive contradiction candidate-generation pass is an $O(n^2)$ all-pairs cosine comparison, which becomes expensive on large corpora. Above a threshold, CAIRN substitutes an in-process approximate-nearest-

neighbor index using random-projection locality-sensitive hashing (SimHash), which compares only vectors likely to be neighbors, entirely in-process with no external vector database. On a synthetic 20,000-vector, 768-dimension benchmark the index completed candidate-generation in roughly two seconds versus an extrapolated brute-force baseline of minutes — an approximate ***~76× reduction, measured on a 2,000-pair subset and scaled*** (the benchmark labels this figure "extrapolated"; it should never be quoted without that qualifier) — at 98.3% recall and 100% precision of returned pairs (every returned pair is exact-cosine-verified before it is kept). An exact-parity test asserts the ANN path recovers the same contradiction candidates as the brute-force scan on a planted set, so the scale layer changes performance, not results.

---

8. Access Control, Deployment, and Residency

Authorization is role-based — viewer, analyst, admin — with deny-by-default on unmapped routes. Authentication accepts API keys and, for enterprise SSO, an OIDC-ready `verifyBearer(token)` hook that validates the firm's identity provider's token and returns a principal with no code change. In the single-user local default (no keys configured) the system runs in open mode; configuring keys or a verifier locks it to authenticated, role-checked access. The server binds loopback by default; LAN exposure is explicit opt-in. Deployment is a non-root container with no dependencies to patch and writable state confined to a single directory; an ungated liveness endpoint supports orchestration probes while every other route is gated in closed mode. Because models and corpus can run fully local, the system is air-gap capable: nothing leaves the perimeter unless an external model endpoint is deliberately configured.

---

9. Compliance and Controls Mapping

CAIRN is a **control the customer operates**, not a certification — certification is a property of the operated system, not of a component. The system maps to common control families as follows: *data residency and confidentiality* (local-by-default, air-gap capable); *access control* (RBAC, API keys, OIDC-ready, deny-by-default); *audit-trail and evidence integrity* (the hash-chained receipt ledger and its verification endpoint, plus an auditor evidence-bundle export combining the current integrity posture, the full sealed chain, and its verification); *AI-output traceability* (cite-or-refuse with per-answer receipts, relevant to model-risk hygiene); and *ongoing monitoring* (the deterministic integrity report and new-defect surveillance). The control-mapping document is cited to code and makes no false certification claims.

---

10. Verification Methodology

The system is covered by 53 automated tests (`node --test`), spanning the index, hybrid search, the deterministic audit, integrity scoring, the receipt ledger, and the connector/ingestion path. Consistent with the JourdanLabs three-leg discipline, the results were **independently re-executed rather than accepted on assertion**: an isolated instance was booted in closed mode and subjected to an authorization probe battery

(unauthenticated 401; wrong-key 401; viewer denied admin routes; health ungated and non-leaking; a nonsense question refused and receipted as `REFUSED_UNGROUNDED`). The receipt ledger was then attacked three ways on a disposable copy — a payload edit, a chain-link edit, and a line deletion — and each tampering was caught with the correct cause. The scale layer's recall and precision were reproduced, and the ANN-versus-brute-force parity confirmed.

---

11. Limitations and Honest Boundaries

These are stated deliberately; a smoothed limitations section would contradict the system's own thesis.

- **Customer-environment last mile.** The connectors, SSO hook, and deployment are config-driven and coded to real vendor APIs and fixture-tested, but a live Confluence/SharePoint pull needs the customer's tenant and token, SSO needs the customer's identity provider, and in-perimeter deployment and any external certification are the customer's. These cannot be, and are not claimed to have been, exercised outside that environment.
- **Scaffolded, not wired.** A per-request access log and an interval surveillance scheduler are implemented but not yet active; access is enforced but per-request access logging is not yet persisted, and surveillance runs on demand rather than on a background loop.
- **Ledger semantics.** The chain assumes a single writer per file, and the `ts` field is outside the hash (§4.4).
- **Contradiction detection yields candidates.** Similarity surfaces candidates; only model adjudication yields a verdict, and adjudication quality is bounded by the configured model.
- **Not certified.** CAIRN generates the evidence a SOC 2 / model-risk / residency review requires; it is not itself certified, and it does not represent otherwise.

---

12. Related Work

Enterprise search platforms optimize findability and rank documents well, but do not verify sources, do not score the corpus for internal conflict, and do not receipt answers. Document-AI assistants optimize fluency and typically operate cloud-side; fluency is not provenance, and a correct-sounding answer from the wrong document remains wrong with no artifact to catch it. Retrieval-augmented-generation plugins ground answers in retrieved chunks but stop there: no pre-generation refusal discipline, no corpus-level integrity score, and no hashed receipt. To our knowledge, no widely deployed system both scores a corpus for internal contradiction and emits a verifiable, per-source-hashed receipt for each answer and refusal.

---

13. Conclusion

CAIRN reframes "chat with your documents" from an answering problem into a provenance-and-integrity problem, and takes the position that in a regulated environment the correct output contract is *proof or refusal*. It combines a refusal-first confidence gate, a tamper-evident hash-chained receipt on every answer and refusal,

and a corpus-level integrity model that separates cheap structural defects from model-confirmed contradictions — implemented with zero runtime dependencies, local-first, and auditable end-to-end. Determinism, receipts, and refusal are structural properties of the implementation rather than aspirational features, which is what makes the system suitable as a control inside a regulated perimeter. Where a capability depends on the customer's own environment, that boundary is named rather than blurred — because a knowledge-integrity tool that overstated its own status would fail its first and most important test.

References

[1] JourdanLabs. *CAIRN — Positioning.* Internal document, 2026. [2] JourdanLabs. *CAIRN — Compliance & Controls Mapping.* Repository document (`docs/COMPLIANCE.md`), 2026. [3] JourdanLabs. *CAIRN Enterprise — Build Plan.* Repository document (`docs/ENTERPRISE-PLAN.md`), 2026. [4] Robertson, S., Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval, 2009. [5] Charikar, M. *Similarity Estimation Techniques from Rounding Algorithms.* STOC, 2002. (Random-projection LSH / SimHash.)

Leland Jourdan II — Founder & Chief Architect, JourdanLabs · `leland@jourdanlabs.com` *Built by JourdanLabs. We build tools that refuse to bluff.*